

Chap9-Exercises_LuisCorreia-745724_v1

October 25, 2025

1 MAP5935 - Statistical Learning (Chapter 9 - Support Vector Machines)

Prof. Christian Jäkel

<https://www.statlearning.com/>

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

1.1 *Conceptual Exercises*

1.1.1 (1) This problem involves hyperplanes in two dimensions.

1.1.2 (a) Sketch the hyperplane $1 + 3X_1 - X_2 = 0$. Indicate the set of points for which $1 + 3X_1 - X_2 > 0$, as well as the set of points for which $1 + 3X_1 - X_2 < 0$.

1.1.3 (b) On the same plot, sketch the hyperplane $-2 + X_1 + 2X_2 = 0$. Indicate the set of points for which $-2 + X_1 + 2X_2 \geq 0$, as well as the set of points for which $-2 + X_1 + 2X_2 \leq 0$.

```
[2]: # -----
# Functions (hyperplanes)
# -----
def f1(x1, x2):
    return 1 + 3*x1 - x2

def f2(x1, x2):
    return -2 + x1 + 2*x2

# Grid
x1_min, x1_max = -3, 3
```

```

x2_min, x2_max = -3, 3

xx1, xx2 = np.meshgrid(
    np.linspace(x1_min, x1_max, 400),
    np.linspace(x2_min, x2_max, 400),
)
zz1 = f1(xx1, xx2)
zz2 = f2(xx1, xx2)

# Lines
x1_line = np.linspace(x1_min, x1_max, 400)
x2_line_f1 = 1 + 3 * x1_line
x2_line_f2 = 1 - 0.5 * x1_line

# Intersection
intersection = (0, 1)

# -----
# Plot with color shading (no hatching)
# -----

fig, ax = plt.subplots(figsize=(8, 8))

# Region shading for f1
ax.contourf(
    xx1, xx2, zz1,
    levels=[-np.inf, 0, np.inf],
    colors=["#c6dbef", "#08306b"], # light blue / dark blue
    alpha=0.15
)

# Region shading for f2
ax.contourf(
    xx1, xx2, zz2,
    levels=[-np.inf, 0, np.inf],
    colors=["#fee0d2", "#99000d"], # light red / dark red
    alpha=0.15
)

# Hyperplane lines
ax.plot(x1_line, x2_line_f1, color="tab:blue", linewidth=2, label=r"$1 + 3X_1 - X_2 = 0$")
ax.plot(x1_line, x2_line_f2, color="tab:orange", linewidth=2, label=r"$-2 + X_1 + 2X_2 = 0$")

# Intersection point
ax.scatter(*intersection, color="red", s=60, zorder=5)

```

```

ax.text(intersection[0] + 0.05, intersection[1] + 0.05, "(0,1)", color="red",
        ↪fontsize=9)

# Reference point (origin)
ax.scatter(0, 0, color="black", s=40, zorder=4)
ax.text(0.1, -0.15, "(0,0)", color="black", fontsize=9)

# Region labels
ax.text(-2.5, -2.5, r"$1 + 3X_1 - X_2 > 0$", color="tab:blue", fontsize=9)
ax.text(1.5, 3.5, r"$1 + 3X_1 - X_2 < 0$", color="tab:blue", fontsize=9)
ax.text(2.0, 2.3, r"$-2 + X_1 + 2X_2 > 0$", color="tab:orange", fontsize=9)
ax.text(-2.5, -0.8, r"$-2 + X_1 + 2X_2 < 0$", color="tab:orange", fontsize=9)

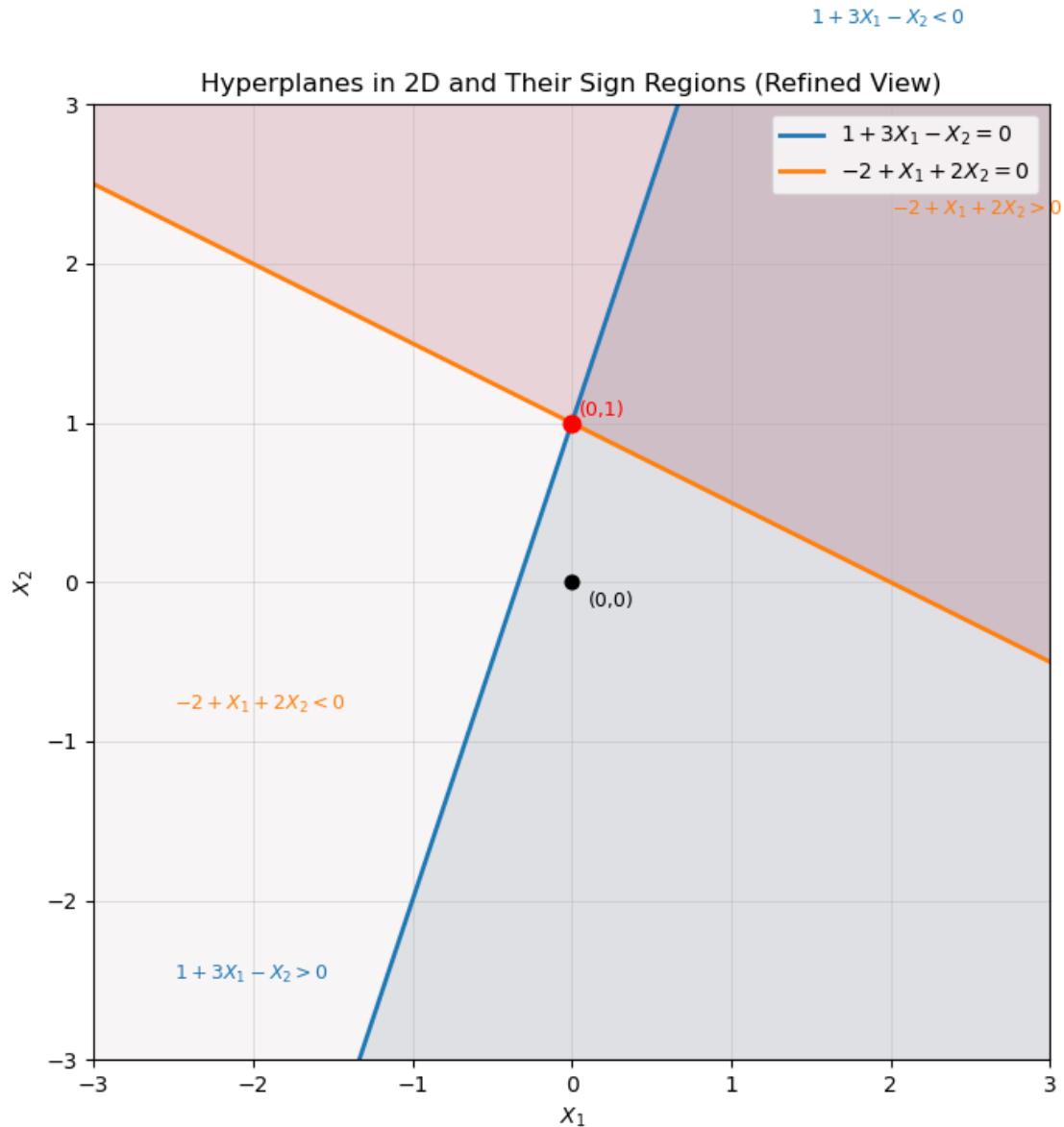
# -----
# Formatting
# -----
ax.set_xlim(x1_min, x1_max)
ax.set_ylim(x2_min, x2_max)
ax.set_aspect("equal", adjustable="box")

ax.set_xlabel(r"$X_1$")
ax.set_ylabel(r"$X_2$")
ax.set_title("Hyperplanes in 2D and Their Sign Regions (Refined View)")

ax.legend(loc="upper right", frameon=True)
ax.grid(True, linewidth=0.5, alpha=0.5)

plt.show()

```



Comments:

The plot shows the two lines representing the hyperplanes:

- $1 + 3X_1 - X_2 = 0$ (blue) and $-2 + X_1 + 2X_2 = 0$ (orange).
- They intersect at $(0, 1)$, dividing the plane into four distinct regions.
- The area **below the blue line** corresponds to $1 + 3X_1 - X_2 > 0$, while the **above** region indicates $1 + 3X_1 - X_2 < 0$.
- For the orange line, the **above** region represents $-2 + X_1 + 2X_2 > 0$, and the **below** region corresponds to $-2 + X_1 + 2X_2 < 0$.
- The point $(0, 0)$ lies in the region where $1 + 3X_1 - X_2 > 0$ and $-2 + X_1 + 2X_2 < 0$, confirming that the hyperplanes partition the plane into distinct half-spaces according to the sign of each

equation.

1.2 Applied Exercises

- 1.2.1 (4) Generate a simulated two-class data set with 100 observations and two features in which there is a visible but non-linear separation between the two classes. Show that in this setting, a support vector machine with a polynomial kernel (with degree greater than 1) or a radial kernel will outperform a support vector classifier on the training data. Which technique performs best on the test data? Make plots and report training and test error rates in order to back up your assertions.

```
[3]: # =====
# Exercise (4) - Nonlinear Separation & SVM Comparison
# =====

# 1) Generate nonlinear two-class data (clearly not linearly separable)
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 2) Fit SVM models with different kernels
svm_linear = SVC(kernel="linear", C=1).fit(X_train, y_train)
svm_poly   = SVC(kernel="poly", degree=3, C=1).fit(X_train, y_train)
svm_rbf    = SVC(kernel="rbf", gamma=1, C=1).fit(X_train, y_train)

# 3) Compute training & test accuracies
models = {
    "Linear SVC": svm_linear,
    "Polynomial (degree=3)": svm_poly,
    "RBF kernel": svm_rbf
}

results = []
for name, mdl in models.items():
    train_acc = accuracy_score(y_train, mdl.predict(X_train))
    test_acc = accuracy_score(y_test, mdl.predict(X_test))
    results.append((name, train_acc, test_acc))

results_df = pd.DataFrame(results, columns=["Model", "Train Accuracy", "Test_
↪Accuracy"])
display(results_df)

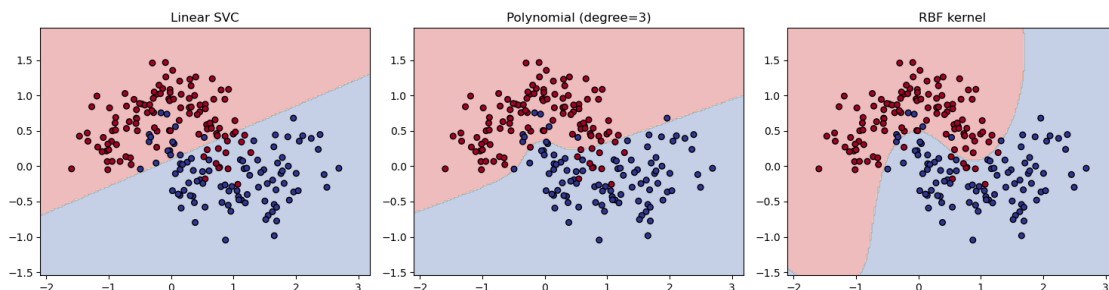
# 4) Plot decision boundaries for each model
```

```
def plot_decision_boundary(model, X, y, ax, title):
    h = 0.02
    x_min, x_max = X[:,0].min()-0.5, X[:,0].max()+0.5
    y_min, y_max = X[:,1].min()-0.5, X[:,1].max()+0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    ax.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu, alpha=0.3)
    ax.scatter(X[:,0], X[:,1], c=y, cmap=plt.cm.RdYlBu, edgecolor="k", s=30)
    ax.set_title(title)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (name, mdl) in zip(axes, models.items()):
    plot_decision_boundary(mdl, X_train, y_train, ax, name)

plt.tight_layout()
plt.show()
```

	Model	Train Accuracy	Test Accuracy
0	Linear SVC	0.838095	0.888889
1	Polynomial (degree=3)	0.885714	0.888889
2	RBF kernel	0.909524	0.911111



Comments

- The **linear SVM** (accuracy 0.84 train / 0.89 test) produces a straight decision boundary that clearly fails to capture the curved structure of the data. It separates the classes roughly but leaves many points on the wrong side of the boundary, evidencing **underfitting**.
- The **polynomial kernel (degree = 3)** slightly improves the boundary's flexibility (0.89 train / 0.89 test), bending along the data structure but still missing the complex curvature between the two clusters. Its performance remains close to the linear model.
- The **RBF kernel** achieves the best fit (0.91 train / 0.91 test), with a smooth, nonlinear boundary that closely follows the true separation between the two moon-shaped classes. The nearly identical training and test accuracies indicate **good generalization** without overfitting.

Conclusion:

The results confirm that kernel-based SVMs outperform the linear SVC on nonlinear problems. While the polynomial kernel provides minor improvement, the **RBF kernel best captures the nonlinear pattern**, offering the lowest classification error and the most accurate decision boundary.